Project Report: TrustVerify CLI Tool

Student ID 1: 230208966

Student Name 1: Artemii Fedorov

Project Report: TrustVerify CLI Tool

Team Name: Darwin Team **Date:** 29/03/2026

1. Project Objective

The goal of this project was to develop a Python-based Command Line Interface (CLI) tool that ensures both the integrity and the origin of files within a directory. The system uses SHA-256 for local integrity checks and RSA digital signatures to provide authenticity and non-repudiation.

2. Implementation Overview

The tool implements the following core features:

- **Hashing (SHA-256):** Generates unique digital fingerprints for any file type (Text, PDF, Image).
- **Manifest Generation:** Scans a directory and creates a `metadata.json` file containing filenames and their respective hashes.
- **Digital Signature (RSA):** Generates a 2048-bit Public/Private key pair. The sender signs the SHA-256 hash of the manifest using their Private Key.
- **Verification:** A receiver uses the sender's Public Key to verify that the manifest is unaltered and originated from the legitimate sender.

3. Theoretical Analysis

3.1. Why hashing alone isn't enough to prove identity

Hashing is a powerful tool for ensuring **data integrity**. It allows a user to detect if a file has been modified. However, hashing fails to prove **identity** (Authenticity).

In a real-world scenario, an attacker could intercept a file, modify its content, calculate a new SHA-256 hash, and update the `metadata.json` file accordingly. Without a digital signature, the receiver has no way of knowing if the manifest was created by the original sender or by an intruder. Hashing only proves that the file matches the manifest, but it does not prove who created the manifest.

3.2. How the Private/Public key relationship ensures non-repudiation

Digital signatures solve the identity problem through asymmetric cryptography. This ensures **non-repudiation**—the inability of a sender to deny the validity of their signature.

- **The Private Key:** Used only by the sender to sign the data. Since the key is kept secret, only the owner can generate a valid signature.
- **The Public Key:** Shared with everyone to verify the signature.
- **The Link:** Because the signature can only be successfully verified with the sender's specific public key, it serves as cryptographic proof that the sender (the holder of the private key) was the one who performed the signing. This creates a permanent, verifiable link between the sender's identity and the data.